SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I **Ramchnadran V(19221108)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ***Ramchandran V***

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic iii. Analyze the impact of heuristic accuracy on both algorithms iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation ii. Explain how backtracking search works in this scenario iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency iv. Analyze real-world challenges and justify the best strategy for scalability

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem ii. Analyze the computational challenges of large game trees iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. AI-Powered Drone for Emergency Medicine Delivery

i. Explain how Greedy Best-First Search behaves in this scenario and discuss its strengths and weaknesses under uncertainty.

Greedy Best-First Search selects the path that appears closest to the destination based on a heuristic function.

- Working: The drone uses straight-line distance to select the next location that seems closest to the medicine delivery point.
- Strengths: It is fast, simple, and often finds a route quickly when time is limited.
- Weaknesses: It considers only the heuristic value and ignores the total path cost already travelled. It may select a route with blocked roads, bad weather, or dangerous terrain.
- Under uncertainty: Incomplete information about blocked paths may cause the drone to choose an unsafe or inefficient route.

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic.

A* Search selects a path using both the actual cost and the estimated remaining cost.

The evaluation function is:

$$f(n)=g(n)+h(n)$$

Where:

- $g(n)$ = actual cost from the starting point to node n.
- $h(n)$ = estimated cost from node n to the destination.
- $f(n)$ = estimated total cost of the path.

The drone considers travel time, terrain difficulty, and weather-related costs along with the estimated distance to the destination. It generally finds an optimal route when the heuristic is admissible and the search conditions satisfy the algorithm's assumptions. When conditions change, the route must be updated or searched again.

iii. Analyze the impact of heuristic accuracy on both algorithms.

- In Greedy Best-First Search, an accurate heuristic helps the drone move toward the destination quickly. An inaccurate heuristic may direct it toward a blocked or costly route.
- In A*, an accurate heuristic improves search efficiency and helps identify a low-cost route.
- If the heuristic overestimates the remaining cost, A* may lose its optimality guarantee.

- A straight-line distance heuristic may be less reliable when weather, terrain, and obstacles significantly affect travel time.

iv. Discuss how dynamic changes (blocked paths) affect search performance.

Blocked paths may make a previously selected route unusable.

- The drone must detect blocked paths using available real-time updates.
- It must remove unusable routes from consideration and calculate an alternative path.
- Greedy Best-First Search may quickly choose a new route but may ignore its total cost.
- A* can compare alternative routes using actual and estimated costs, but repeated searches require additional processing time.
- Frequent updates and safe route replanning are necessary for reliable medicine delivery.

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

Recommendation: A Search with real-time route replanning.*

A* is suitable because it considers both the cost already incurred and the estimated remaining cost. It can find a minimum-cost route under appropriate heuristic conditions. The drone should update its map when new information arrives and avoid blocked or unsafe paths. For strict real-time operation, an incremental replanning method may further reduce computation. Safety constraints must take priority over distance alone.

2. Smart City Traffic Signal Optimization

i. Explain how Hill Climbing would work in this scenario and identify its limitations.

Hill Climbing starts with an initial set of traffic signal timings and searches for a better neighbouring solution.

Working:

1. Initialize signal timings for all intersections.
2. Calculate the waiting time, fuel consumption, and congestion.
3. Change one or more signal timings slightly.
4. Evaluate the new solution.
5. Accept the change if it improves the objective.
6. Repeat until no better neighbouring solution is found.

Limitations:

- It can get stuck in a local optimum.
- It may fail to explore better solutions that require temporary worsening.

- It is sensitive to the initial solution.
- It may struggle with rapidly changing traffic conditions.

ii. Discuss local maxima, plateaus, and ridges with real-world interpretation.

- Local optimum: A signal timing arrangement improves one junction but causes congestion at nearby junctions. No small local change appears beneficial, although a better overall arrangement exists.
- Plateau: Many timing combinations produce almost the same traffic performance. The algorithm cannot easily identify which direction will improve the result.
- Ridge: Improvement requires coordinated changes across several connected intersections. Changing only one signal at a time may not find the better solution.

These problems can prevent Hill Climbing from reaching the best overall traffic arrangement.

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.

Simulated Annealing:

- Starts with an initial solution and makes small random changes.
- Usually accepts improvements.
- Sometimes accepts a worse solution, especially at higher temperatures, to escape local optima.
- Gradually reduces the temperature and becomes more selective.
- Can explore different traffic timing arrangements without always getting stuck in a local optimum.

Genetic Algorithm:

- Creates a population of possible signal timing solutions.
- Evaluates each solution using the objective function.
- Selects better solutions for reproduction.
- Uses crossover to combine solutions and mutation to introduce variation.
- Repeats the process over several generations to find improved timings.

Both approaches can explore a wider search space than basic Hill Climbing.

iv. Recommend the best approach and justify your choice based on scalability and adaptability.

Recommendation: Genetic Algorithm with periodic or event-driven optimization.

A Genetic Algorithm can optimize multiple intersections together and explore a large search space using a population of candidate solutions. It is suitable for balancing waiting time, fuel consumption, and congestion. The system can update its population when traffic patterns change. For real-time deployment, the algorithm should use bounded computation time, preserve safety constraints, and be combined with a fast local adjustment method. Simulated Annealing may be preferable if implementation simplicity and lower computational overhead are more important than population-based exploration.

3. Autonomous Rover for Mars Exploration

i. Explain why this problem requires an online search agent instead of offline search.

An online search agent is suitable because the rover explores unknown terrain and cannot obtain a complete map before starting its journey.

- It makes decisions using currently available information.
- It discovers new terrain while moving.
- It updates its route when it finds obstacles or hazards.
- It adapts its actions according to new sensor information.
- It supports real-time decision-making in an unknown environment.

Offline search requires a sufficiently known state space and model before planning. Therefore, online search is more appropriate for the rover's uncertain environment.

ii. Analyze the challenges of partial observability and dynamic environments.

Partial observability:

- The rover cannot observe the entire environment at once.
- Sensors may provide incomplete or noisy information.
- Hidden craters and rocks may create unexpected risks.
- The rover must make decisions with limited knowledge.

Dynamic environment:

- New obstacles may be discovered during movement.
- Terrain conditions may change.
- Communication delays may prevent immediate assistance from Earth.
- The rover must update its route and avoid unsafe movements.

These challenges require continuous sensing, planning, and decision-making.

iii. Describe how the rover builds and updates its internal model.

The rover builds an internal model using sensors, cameras, and navigation data.

1. Collect information about the surrounding terrain.
2. Identify obstacles, craters, rocks, and safe areas.
3. Represent the discovered terrain as a map or graph.
4. Estimate the cost and risk of possible movements.
5. Select a suitable path toward the target.
6. Move through the selected path while monitoring the environment.
7. Update the map when new information is received.
8. Replan when the current route becomes unsafe or unsuitable.

This process helps the rover gradually improve its understanding of the environment.

iv. Compare online search with uninformed and informed search strategies.

Feature	Online Search	Uninformed Search	Informed Search
Knowledge	Learns during exploration	Uses the problem definition without a heuristic	Uses heuristic information
Environment	Suitable for unknown environments	Usually searches a known problem representation	Usually searches a known problem representation
Guidance	Uses observations and available knowledge	No heuristic guidance	Uses estimated distance or cost
Adaptability	Can replan as new information arrives	Standard methods do not automatically adapt	Standard methods do not automatically adapt
Example	Online agent navigating unknown terrain	Breadth-First Search	A* Search

Online search describes how an agent acts while learning about its environment. Uninformed and informed search describe how search choices are guided; they can also be incorporated into online agents.

v. Justify the most suitable approach considering uncertainty, adaptability, and safety.

Recommendation: Online search combined with heuristic-based path planning.

The rover should use an online agent that collects sensor information, updates its internal map, and replans when obstacles are discovered. A* can help estimate efficient routes when a usable map and cost model are available. The rover should also apply safety constraints to avoid hazardous terrain and preserve energy for essential tasks.

This combination supports exploration, adaptability, and safe movement without requiring complete environmental knowledge in advance.

4. University Examination Timetable Using Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation.

1. Variables

Each examination is represented by a variable.

Example:

- \(E_1\) = Mathematics examination.
- \(E_2\) = Artificial Intelligence examination.
- \(E_3\) = Database Management examination.

Each variable represents the time slot and examination hall assigned to a subject.

2. Domains

The domain contains the possible values for each variable.

Example:

- Time slots: Monday morning, Monday afternoon, Tuesday morning.
- Examination halls: Hall A, Hall B, Hall C.

An examination can be assigned only to an available time slot and hall.

3. Constraints

- No student conflict: A student must not have two examinations at the same time.
- Hall capacity: The assigned hall must accommodate all students taking the examination.
- Hall availability: A hall cannot host two examinations simultaneously.
- Subject order: A prerequisite examination must occur before a dependent examination when required.
- Faculty availability: Examinations must be scheduled when the required faculty members are available.
- Special accommodations: Students requiring special arrangements must be assigned suitable halls and time slots.

The goal is to find a timetable that satisfies all hard constraints.

ii. Explain how backtracking search works in this scenario.

Backtracking Search assigns values to variables one at a time and checks whether each assignment satisfies the constraints.

Steps:

1. Select an examination that has not been scheduled.
2. Choose an available time slot and hall.
3. Check whether the assignment satisfies all constraints.
4. If valid, assign the time slot and continue.
5. If invalid, try another available combination.
6. If no combination works, return to the previous examination.
7. Change the previous assignment and continue searching.
8. Stop when a valid timetable is found or all possibilities have been explored.

This method avoids keeping assignments that violate the scheduling rules.

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency.

Constraint propagation reduces the number of possible assignments before the entire timetable is completed.

Forward checking:

- After assigning a time slot to an examination, remove conflicting time slots from the domains of related examinations.
- If an examination has no remaining valid time slot, backtrack immediately.
- Check hall availability and student conflicts before making further assignments.
- Reduce unnecessary search and improve efficiency.

For example, if Mathematics is scheduled for Monday morning, another examination involving the same students cannot be scheduled at that time.

Forward checking helps detect conflicts early and reduces the search space.

iv. Analyze real-world challenges and justify the best strategy for scalability.

Real-world challenges:

- A large number of students and subjects.
- Limited examination halls and seating capacity.
- Faculty availability and scheduling conflicts.
- Different examination durations.
- Special accommodations and student requirements.
- Last-minute changes in course registrations or hall availability.

Recommended strategy: Backtracking Search with constraint propagation and heuristics.

The system should use forward checking, the Minimum Remaining Values (MRV) heuristic, and degree-based variable selection.

- MRV selects the examination with the fewest available choices.

- Degree-based selection prioritizes examinations that constrain the largest number of other examinations.
- Forward checking detects conflicts early.
- Backtracking allows the system to recover from invalid assignments.

For large universities, these methods can be combined with local-search optimization to improve timetable quality and scalability.

5. Strategic Game AI for Real-Time Decision Making (Minimax and Alpha-Beta Pruning)

i. Explain how the Minimax algorithm models this problem.

Minimax is a decision-making algorithm used in competitive games where two players have opposing objectives.

- The MAX player represents the AI opponent and tries to maximize its score.
- The MIN player represents the human opponent and tries to minimize the AI's score.
- The algorithm builds a game tree containing possible moves and resulting game states.
- It evaluates terminal states or cutoff states using a scoring function.
- MAX selects the highest available value, while MIN selects the lowest.
- The AI chooses the move that gives the best guaranteed outcome, assuming the opponent plays optimally.

Minimax helps the AI make strategic decisions by considering the possible responses of the human player.

ii. Analyze the computational challenges of large game trees.

Large game trees create several computational challenges:

- High branching factor: Each game state may have many possible moves.
- Deep search: The AI may need to explore many future moves.
- High memory usage: Storing game states and their results requires memory.
- Time constraints: Real-time games require decisions within a limited time.
- Changing game states: Opponents may make unexpected moves.
- Expensive evaluation: Complex game states may require considerable processing.

The time complexity of Minimax is approximately:

\[O(b^d) \]

Where:

- \(b\) = average number of possible moves per state.

- d = search depth.

Therefore, searching every possible move becomes impractical for large games.

iii. Explain how Alpha-Beta Pruning improves efficiency with detailed reasoning.

Alpha-Beta Pruning improves Minimax by eliminating branches that cannot affect the final decision.

It maintains two values:

- Alpha (α): The best value MAX can guarantee so far.
- Beta (β): The best value MIN can guarantee so far.

A branch can be pruned when:

$$\alpha \geq \beta$$

Working steps:

1. Start with the root game state.
2. Explore the possible moves using Minimax.
3. At MAX nodes, update alpha with the highest value found.
4. At MIN nodes, update beta with the lowest value found.
5. When alpha becomes greater than or equal to beta, stop exploring the remaining branches of that node.
6. Continue evaluating the remaining relevant branches.
7. Select the best move based on the resulting Minimax value.

Example:

Suppose MAX has two possible moves, A and B.

- Move A has a final value of 5.
- While examining move B, MIN finds an option with a value of 3.
- Since MIN can force a value no greater than 3 for that branch, MAX will prefer the already available value of 5.
- The remaining branches under B can be pruned when the alpha-beta condition is met.

Alpha-Beta Pruning produces the same optimal decision as Minimax when applied correctly, but it can examine far fewer nodes. With ideal move ordering, its time complexity can improve to approximately $O(b^{d/2})$.

iv. Design an evaluation function and justify the factors considered.

An evaluation function estimates how favourable a game state is for the AI when searching cannot continue to a terminal state.

A general evaluation function can be written as:

\[E(s)=w_1M+w_2P+w_3R+w_4O-w_5D \]

Where:

- \(M\) = material or resource advantage.
- \(P\) = positional advantage.
- \(R\) = territory or objective control.
- \(O\) = offensive potential.
- \(D\) = defensive vulnerability.
- \(w_1,w_2,w_3,w_4,w_5\) = weights assigned to the factors.
- \(s\) = current game state.

Justification:

- Material or resource advantage measures the AI's available units and resources compared with the opponent.
- Positional advantage evaluates the strategic strength of the AI's positions.
- Territory or objective control measures progress toward winning.
- Offensive potential estimates the ability to attack the opponent.
- Defensive vulnerability measures the risk of losing units or important objectives.

The weights should be adjusted according to the rules and objectives of the particular game. A higher evaluation score should represent a more favourable position for the AI.

v. Recommend a strategy for balancing optimality and real-time performance.

Recommendation: Minimax with Alpha-Beta Pruning, heuristic evaluation, and iterative deepening.

- Alpha-Beta Pruning reduces unnecessary game-tree exploration.
- Heuristic evaluation estimates the value of non-terminal states.
- Iterative deepening searches progressively greater depths until the time limit is reached.
- Move ordering helps promising moves get explored earlier, improving pruning.
- Transposition tables can store previously evaluated game states to avoid repeated work.
- A fixed computation budget ensures that the AI can return a move within the required time.

This approach provides strong decisions while balancing computational cost and real-time responsiveness.

Conclusion

The five scenarios demonstrate how AI search and optimization techniques solve complex real-world problems. Greedy Best-First Search and A* support route planning, Hill Climbing and

Genetic Algorithms help optimize traffic signals, online search enables autonomous exploration, CSP solves examination timetabling, and Minimax with Alpha-Beta Pruning supports strategic game decisions. Selecting the right algorithm depends on the problem's uncertainty, constraints, computational cost, safety requirements, and response time.